

Лабораторная работа

Однослойный перцептрон: реализация и обучение

Выполнил работу: Айвазян Арман Б25-527

Ссылка на гит: https://github.com/armanncho/LACM_Laba2.git

Цель работы

- Реализовать алгоритм обучения однослойного перцептрона с нуля (без использования готовых библиотек глубокого обучения).
- Изучить влияние таких параметров, как скорость обучения, количество эпох и размер батча, на процесс обучения.
- Научиться подготавливать данные для нейронных сетей, включая нормализацию и стратификацию.
- Провести анализ полученных результатов и сделать выводы.

Теоретические сведения

Однослойный перцептрон — это простейшая модель искусственной нейронной сети, представляющая собой бинарный классификатор. Математически, для набора входных признаков X и вектора весов w , модель вычисляет линейную комбинацию, к которой прибавляется смещение (bias) b .

Для решения задачи бинарной классификации к полученному результату применяется нелинейная функция активации (в нашем случае — сигмоида), которая сжимает значения в диапазон от 0 до 1:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

где $z = Xw + b$.

Обучение модели происходит за счет минимизации функции потерь с использованием градиентного спуска. Разделяющей границей классов выступает гиперплоскость (в двумерном случае — прямая), описываемая уравнением

$$w^T x + b = 0.$$

Подготовка модели (данных)

Для обучения модели был сгенерирован набор данных для бинарной классификации с двумя числовыми признаками.

Перед подачей данных в модель была выполнена их стандартизация (Z-score) по обучающей выборке, после чего полученные параметры были применены к тестовой выборке. Данные были разделены на обучающую (70%) и тестовую (30%) выборки с использованием стратификации для сохранения исходных пропорций классов.

Реализация модели

Для создания перцептрона был реализован класс, использующий только библиотеку NumPy и стандартные математические функции Python.

Инициализация модели

При инициализации экземпляра класса веса модели w задаются небольшими случайными значениями, а параметр смещения b инициализируется нулём.

Прямой проход

Функция `forward(X)` реализует прямой проход сети. Она вычисляет линейную комбинацию входов и весов, а затем применяет функцию `sigmoid(z)` для получения вероятности принадлежности к положительному классу.

Функции потерь

В качестве функции потерь реализован метод `compute_loss(y_true, y_pred)`, который вычисляет среднее значение бинарной кросс-энтропии.

Формула бинарной кросс-энтропии (BCE):

$$L = -\frac{1}{N} \sum_{i=1}^N (y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i))$$

Обучение

Функция `fit` реализует алгоритм градиентного спуска с использованием мини-батчей. На каждой итерации вычисляются градиенты функции потерь по весам и смещению, после чего параметры обновляются в сторону антиградиента с шагом, равным скорости обучения.

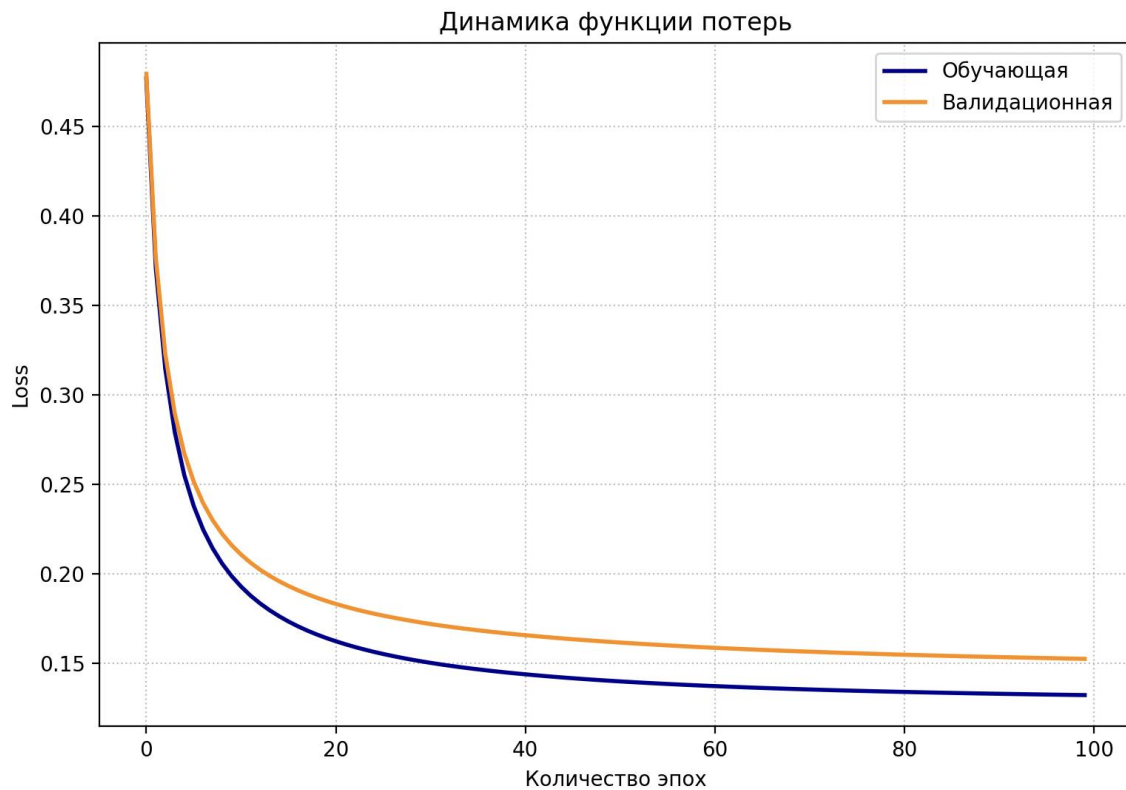
Обучение модели

Модель была обучена на подготовленных данных со следующими базовыми гиперпараметрами:

- Скорость обучения: 0.1
- Количество эпох: 100

- Размер батча: 32

В процессе обучения значения функции потерь (loss) на обучающей и валидационной выборках сохранялись для последующего анализа.



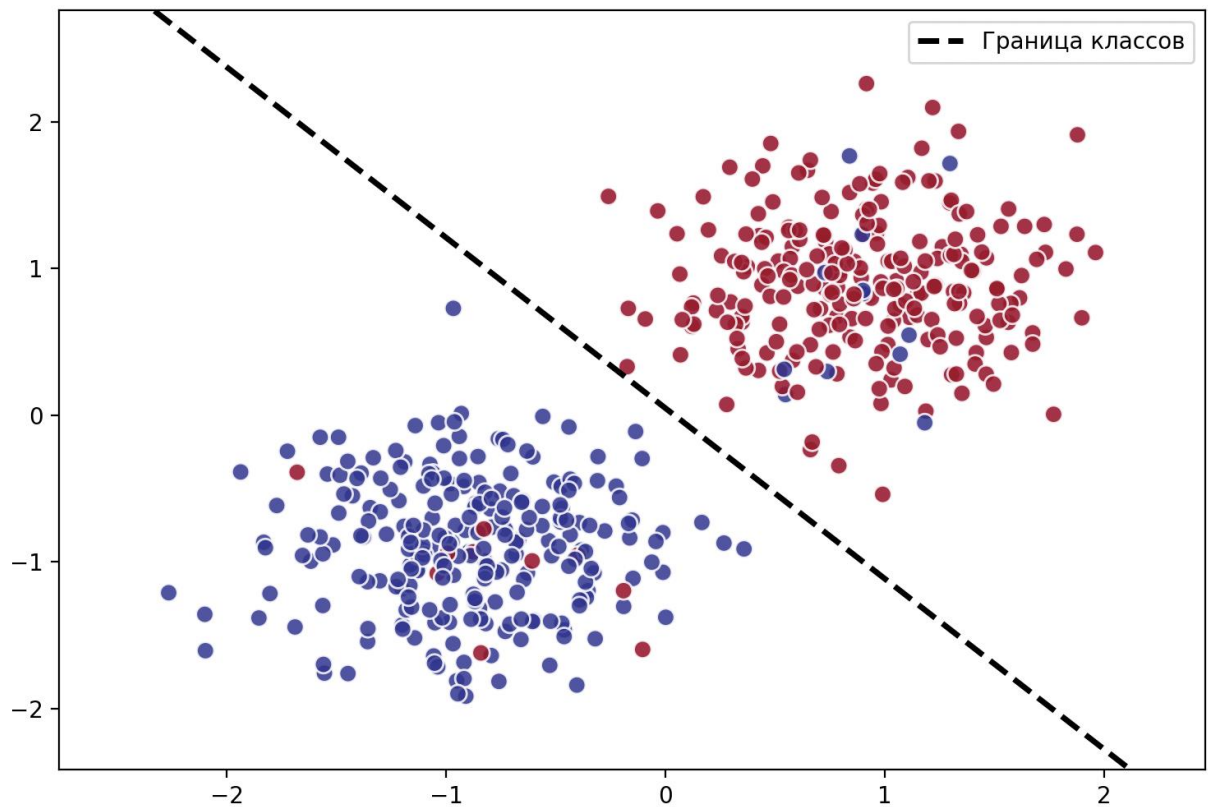
Результаты обучения

После завершения процесса обучения была вычислена итоговая точность (ассигасу) классификации.

Ниже представлены графики изменения функции потерь на обучающей и валидационной выборках в зависимости от эпохи.

Визуализация разделяющей границы

Для наглядной демонстрации работы алгоритма была визуализирована разделяющая граница на фоне точек тестового набора данных. Прямая описывается уравнением $w^T x + b = 0$. Точки по разные стороны от прямой классифицируются моделью как разные классы.



Вывод: В ходе лабораторной работы был успешно реализован и обучен алгоритм однослойного перцептрона. Экспериментально подтверждено, что модель способна находить линейную разделяющую границу для предложенного набора данных. Написанные с нуля функции прямого прохода, вычисления потерь и обратного распространения ошибки позволили на практике понять математический аппарат, стоящий за простейшими нейронными сетями.